An Observational Benchmark for Layer 2 Finality

What has been built, why each piece exists, how the parts fit together, and what every file in the repository is for.

PROJECT	50081 — Enhancing Scalability and Interoperability in Decentralized Layer 2 Blockchain Networks
TRACK	**Scalability** — benchmarking L2 approaches on throughput, latency and cost-efficiency
PROGRAMME	Mitacs Globalink 2026
SUPERVISOR	Prof. Sara Rouhani, TCDT Lab
INTERN	Ujala Kiran
APPROACH	Observational — measure public networks, host nothing
STATUS	32 of 38 tasks complete. Phases A–E done, F and G all but G5; only the write-up and threats-to-validity remain
DATE	11 August 2026, figures as of 08:30 PKT

WHAT THIS DOCUMENT IS

The implementation handbook says *what* to build, task by task. This document records *what was actually built*, what was learned in the process, and why the code looks the way it does. Where a measurement contradicted an expectation, that is written down here rather than quietly fixed — several of those contradictions are the most useful findings so far.

1 · What the project measures

The framework submits a transaction to a Layer 2 network operated by somebody else, then determines three different moments at which that transaction could reasonably be called *final* — and what it cost to reach each one.

LEVEL	FINAL WHEN	WHERE THE TIMESTAMP COMES FROM
$\mathbf{t}_1$ Full trust	The L2 sequencer accepts it	The L2's own RPC — a transaction receipt appears
$\mathbf{t}_2$ Partial trust	Its batch is posted to Ethereum	An Ethereum L1 transaction sent by the rollup operator
$\mathbf{t}_3$ Trustless	Its batch is proven, or its challenge window closes	A second Ethereum L1 transaction, later

Two of the three timestamps come from Ethereum, not from the rollup. That single observation is why the project needs no hosted infrastructure: the evidence for partial-trust and trustless finality is public on the L1, sent by the rollup operator, and readable by anyone. It is also what makes every result independently checkable — each row of output carries the Ethereum transaction hashes a reviewer can open for themselves.

Why three levels rather than one

Published L2 benchmarks report "latency" as a single number, which almost always means t_1 — the sequencer said yes. That number is excellent for every rollup and says nothing about the security assumption behind it. Separating the three levels is what allows the comparison this project exists to make: a ZK rollup and an optimistic rollup look similar at full trust and behave very differently once you stop trusting the sequencer.

2 · Where the project stands

PHASE	PURPOSE	TASKS	STATUS
A	Unblock	A1–A5	**Complete**
B	Make it real	B1–B5	**Complete**
C	The contribution	C1–C6	**Complete**
D	Make it defensible	D1–D5	**Complete — minimum viable deliverable**
E	The comparison	E1–E3	**Complete**
F	Data collection	F1–F4	F1, F3 and F4 done; F2 at 28 of 60 repetitions and accumulating hourly
G	Analysis	G1–G5	G1–G4 done; G5 unwritten
H	Write and deliver	H1–H5	Not started

Results obtained so far

1,265 transactions across two architectures, 1,238 of them successful (97.9%), plus read-only observation of both chains' mainnets. 1,237 have settled all the way through t_3, across **eight distinct zkSync batches** — so settlement is no longer a single observation.

Every number in this section is a snapshot. Collection runs hourly and the medians move as it accumulates — OP's partial-trust median fell from 114 s to 34.78 s between two readings an hour apart, as the sample grew from 101 rows to 212. Regenerating the figures re-derives all of them from the raw rows, so the figures and this text are only in step when both are rebuilt.

27 failed, and they are in the data with reasons rather than discarded: 24 timed out with no receipt inside 120 seconds, and 3 were rejected outright by a public endpoint returning HTTP 429. Latency statistics count successes only and the success rate — 96.2% — is reported beside them.

LEVEL	MEDIAN	P95	MIN	MAX
Full trust (t_1)	18.04 s	19.19 s	0.60 s	20.16 s
Partial trust (t_2)	25.47 m	28.30 m	19.48 m	30.42 m
Trustless (t_3)	36.67 m	39.50 m	30.68 m	41.62 m

Broken down by stage, the medians are: 0.30 minutes for the sequencer to accept, **25.17 minutes waiting for the batch to be posted**, and 11.20 minutes from commit to proof. The dominant cost of trustless finality on this rollup is not proving — it is waiting for a batch to seal.

Cost per transaction: **$0.00285965** at an assumed ETH price of $3,800, derived from the actual L1 receipts (commit 0.000495 ETH, proof 0.000164 ETH) divided by the 875 transactions sharing the batch.

HOW MUCH THESE NUMBERS MOVE BETWEEN DAYS

Four matrix cells have now been run on more than one day. Across ten comparisons the median divergence is **12.1%** and the worst is **65.7%**.

The split matters more than the headline. Full-trust latency reproduces almost exactly — 0.1% to 0.9% on small batches — while partial and trustless move by 20% to 66% between days. That is the shape one should expect: t_1 measures a sequencer accepting a transaction, whereas t_2 and t_3 depend on where in a batch interval the submission happened to fall, which is close to arbitrary. Settlement medians are therefore the ones to treat with caution, and the divergence figure is what a reader needs in order to do so.

An earlier version of this document warned that settlement had n = 1. That is no longer true — eight distinct batches have settled — but the successor caution stands: every figure here is a snapshot from an ongoing collection, and the partial-trust ratio in particular read 13.4x, then 44x, then 13.3x as the sample grew. Nothing should be cited without its n.

The cross-architecture comparison

Same workload, same code path, both rollups. This is the result the project exists to produce.

LEVEL	ZKSYNC ERA	KIND	OP SEPOLIA	KIND	RATIO
Full trust (t_1)	35.36 s	observed	25.99 s	observed	comparable
Partial trust (t_2)	25.29 min	observed	1.90 min	estimated	OP **13.3x faster**
Trustless (t_3)	35.82 min	observed	**7.01 days**	derived	OP **281x slower**

Native transfer, medians over 487 and 323 successful transactions. Like for like: an earlier version of this table compared OP's single-transaction run against zkSync's batch of fifty and made the optimistic rollup look thirty times faster at full trust. It is not — with more data the two are now indistinguishable at full trust, 18.90 s against 18.89 s, and the comparison only becomes interesting further down the table.

The partial-trust ratio is worth watching rather than quoting from memory. It read 13.4x, then 44x an hour later, and settled at 13.3x once the sample reached 323 rows. The 44x was a small-sample artefact and would have been embarrassing in a report. Nothing in this table should be cited without the n beside it.

THE INVERSION, AND WHY THE KIND COLUMN IS NOT DECORATION

At full trust the two architectures are indistinguishable in practice. The optimistic rollup is then about thirteen times faster at partial trust, because it posts batches to the L1 far more often - and about two hundred and seventy-five times slower at trustless finality. Which rollup is "faster" depends entirely on which trust assumption was meant, and that is exactly what a single-number latency benchmark cannot show.

But the two t_3 figures are not the same kind of measurement and must never be tabulated as though they were. zkSync's is **observed**: a proof was verified in an Ethereum block whose timestamp we read, and the transaction hash is in the output. OP's is **derived**: it is the moment a challenge window closes, and *nothing happens then* — no transaction, no event, nothing to point at. It is a deadline computed from the output proposal plus the challenge period read from the OptimismPortal.

Likewise OP's t_2 is **estimated** — the batch transaction was matched by block range rather than named by the rollup, because an OP Stack chain exposes no mapping from a transaction to the batch carrying it without decoding the blob. And its L1 cost is reported **unavailable**, because the batch's transaction count is not exposed and there is therefore no denominator to divide by.

Grounded against production

Both mainnets observed read-only, sending nothing and spending nothing.

MEASURE	ZKSYNC ERA MAINNET	OP MAINNET
batch seal / posting interval	30.15 min	4.40 min
L1 commit interval	31.88 min	–
commit → proof	**38.38 min**	–
output proposal interval	–	60.20 min
proof maturity delay	–	**7.00 days**

Two things follow, and both belong in the report. **Mainnet proves about 3.4 times slower than the testnet** — 38.38 minutes against 11.2 — so every testnet trustless figure understates production by a factor that can now be stated rather than gestured at. And **OP Mainnet's proof maturity delay is 7.00 days, identical to OP Sepolia's**, which confirms against production what the testnet reading suggested: this testnet does not shorten its challenge window, though testnets are widely assumed to.

The figures

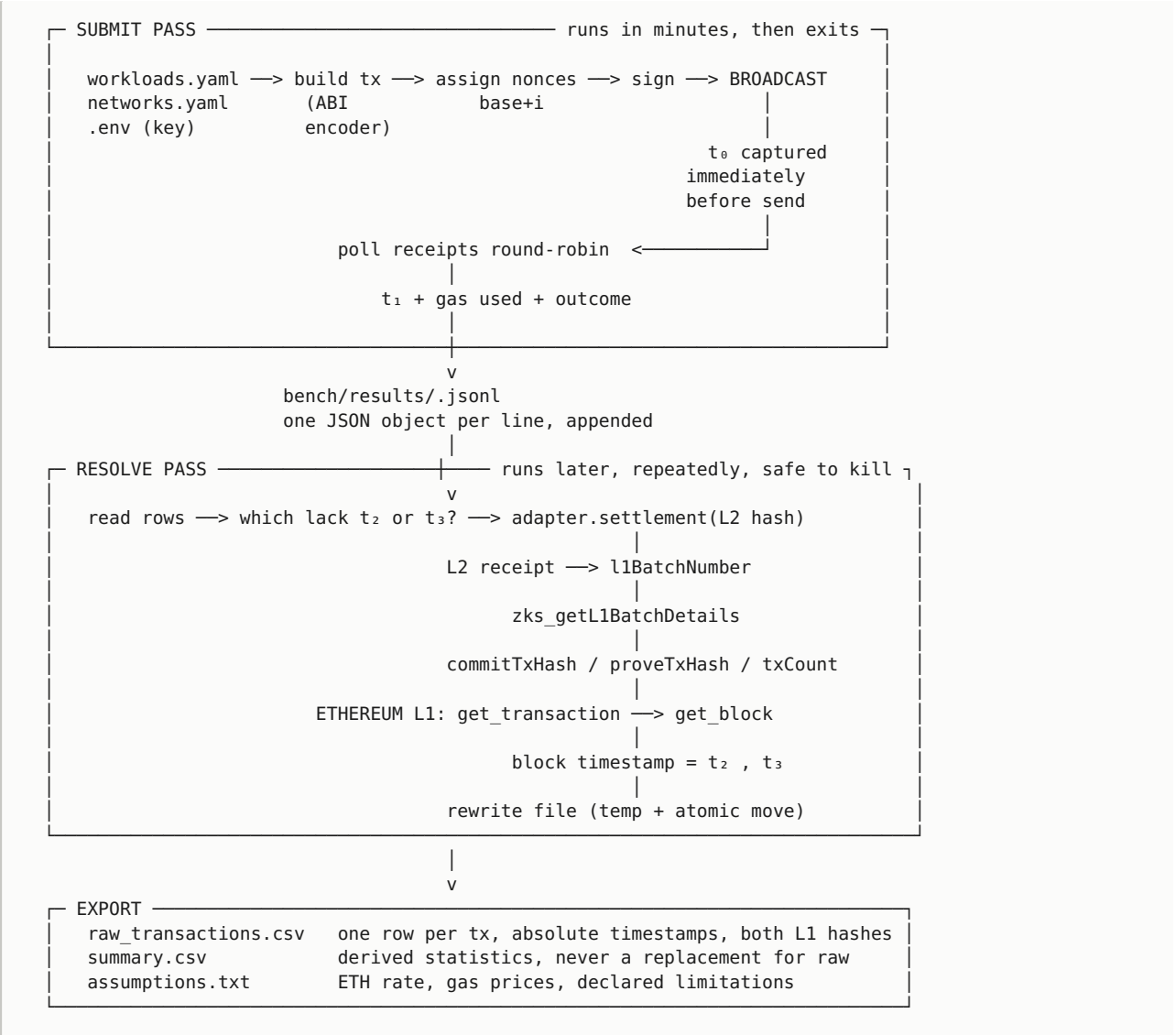
FIGURE	WHAT IT SHOWS
`g1_latency_cdf`	Distributions per finality level, one panel per rollup, shared log axis so the panels are genuinely comparable
`g2_inversion`	The headline. The crossover, drawn solid for observed and dashed for derived
`g3_cost_breakdown`	Where a batch's L1 cost goes, per transaction

Each ships a CSV of the numbers behind it. That is not politeness: three of the palette's slots sit below 3:1 contrast on a white surface, so every figure carries either direct labels or a table view, and a reader should never need to trust a printer or their own colour vision to check a claim.

The cost split on zkSync batch 21623, per transaction: batch posting 74.00%, proof verification 24.83%, blob data availability **1.17%**. Blob DA being nearly free is a property of Sepolia's blob market resting at its price floor, not a result about the rollup, and must not be quoted as a mainnet figure.

3 · How the system flows

The single most important structural decision is that submission and resolution are **two separate programs**. Trustless finality can take an hour. A program that waits synchronously ties up the machine, dies when the laptop sleeps, and loses everything it had collected.



Why this shape

- **t_0 is captured immediately before the broadcast**, after signing and gas estimation. Anything expensive between the two would be silently attributed to the network.
- **The file is the state.** The resolve pass recomputes what is outstanding from disk on every pass and holds nothing between passes, so it can be interrupted at any point and lose nothing.
- **JSON Lines, not a JSON array.** An interrupted append leaves a readable file; a half-written array does not.
- **Only absolute timestamps are stored.** Latencies are computed at export time. An arithmetic mistake becomes a re-export rather than a re-run of the experiment.

4 · Repository layout

```
bench/
    core/          submit, resolve, metrics — network-independent
    adapters/      one file per rollup — the only per-network code
    configs/       one YAML per concern; a run is described entirely by these
    abis/          committed contract ABIs
    contracts/     the workload token and its vendored dependencies
    results/       gitignored JSONL output
    export/        gitignored CSV deliverables
docs/              this handbook and its build script
```

The separation that matters is `core/` against `adapters/`. Everything in `core/` works in terms of a `Settlement` record and never asks which rollup it is talking to. Adding a second architecture means adding one file to `adapters/` and changing nothing else — which is what makes the ZK-versus-optimistic comparison achievable rather than aspirational.

5 · Every file, and what it is for

Entry points — the commands you actually run

`bench/create_wallet.py` A2

Generates the single test account used across every network and writes the private key to `.env`, the address to committed config. One account everywhere keeps records simple, and addresses are compatible across all EVM chains.

`main()`

`bench/check_funds.py` A3 · A4

Answers "where do I actually have money" without opening a wallet, and doubles as the endpoint verification A4 requires. Reports chain ID mismatches and — importantly — **stalled endpoints**: an RPC that answers correctly while serving state frozen weeks in the past. Records balances and faucet provenance into `accounts.yaml` for the report's experimental setup.

`probe · render · next_actions · print_faucets · record`

`bench/send_one.py` A5

A deliberately unabstracted script that sends one transaction and times it. It exists so that when the framework misbehaves later, there is a known-good reference proving that keys, funds, endpoint and chain ID all work together. `--dry-run` signs locally and needs no funds at all.

`build · wait_for_receipt · main`

`bench/deploy_token.py` B1

Compiles `BenchToken.sol` with a pinned solc and deploys it, recording the address into `workloads.yaml` by a targeted line edit that preserves the file's comments. `--dry-run` asks the node whether it will accept the bytecode at all, which is how the question "does this rollup need its own compiler" gets answered by measurement instead of assumption.

`compile_token · record_address · main`

bench/check_workloads.py B1 DONE-WHEN

Builds every workload against a live node and asks for a gas estimate without broadcasting. Estimation is a real simulation against real state, so it catches a corrupt selector, a wrong token address and an unfunded account — which is why it is the acceptance test rather than a local encoding check.

`describe · check · run_network · main`

---

bench/submit_run.py B2 · B3 · B4 · B5

The submit pass. Builds a batch of one workload, numbers it consecutively from a single fetched nonce, broadcasts in index order, polls every receipt round-robin, and appends one JSON object per transaction. Exits as soon as t_1 is known — it never waits for settlement.

`default_run_id · main`

---

bench/resolve_run.py C2–C6

The resolve pass. Reads run files, finds rows still missing t_2 or t_3, asks the rollup where they settled, and writes the files back. Safe to run repeatedly and safe to interrupt. `--watch N` repeats every N seconds and stops itself once nothing is outstanding.

`resolve_file · one_pass · main`

---

bench/export.py D1 · D2 · D3 · D4

Produces the three deliverable files. Computes all latencies here from absolute timestamps, fetches L1 costs once per batch rather than once per transaction, and writes an assumptions block stating the ETH rate, its source, and every declared limitation. `--eth-usd` overrides the configured rate, which is D2's acceptance test.

`latencies · write_raw · batch_costs · summarise_group · main`

---

bench/run_matrix.py F1 · F2 · E3

Drives the experiment matrix. `--plan` prints the checklist and reads completion from the run files themselves rather than a ledger, so the record cannot drift from the data. `--next` runs exactly one cell and refuses a repetition sooner than the configured gap — that refusal is the feature, because five runs back to back measure one moment five times. `--paired` submits to both architectures back to back for the cross-architecture comparison.

`cells · completed · plan · run_next · run_paired`

---

bench/collect.sh F2 · C6

One unattended tick, installed hourly from cron: submit at most one overdue cell, run one resolve pass, exit. Nothing loops. A missed tick means the next one picks up the most overdue cell, and a machine asleep for six hours leaves the matrix six ticks behind rather than corrupted.

---

bench/observe_mainnet.py F4

Reads production batch cadence, proof intervals and challenge parameters without sending anything. For a ZK rollup the batch view gives commit cadence and commit-to-proof directly; for an OP Stack chain the two cadences come from different places — batch postings by scanning the L1 for the batcher, output proposals from the dispute game factory. Every figure is reported with the range it was measured over.

`observe_zk · observe_op · block_at_or_after`

---

bench/analysis/figures.py G1 · G2 · G3

The report's figures, each with a CSV beside it. Observed and derived values are drawn differently — solid against dashed, said in the legend — because a proof verified in an Ethereum block and a deadline that nothing marks are not the same kind of measurement. G3's caption is computed from the data rather than typed, so it cannot come to contradict the chart it sits above.

`figure_cdf` · `figure_inversion` · `figure_cost`

---

bench/check_config.py D5 DONE-WHEN

Finds configuration keys that nothing reads. A setting that is parsed and then ignored is the worst class of benchmark bug: you change it, the run completes, the number is unchanged, and nothing tells you. It found three such keys on first run.

`schema_keys` · `main`

Core — network-independent machinery

bench/core/wallet.py A2

Loads the private key from the environment and confirms it derives the address recorded in committed config. A mismatch means `.env` and the config describe different accounts, which would make every recorded result untraceable.

`load_env` · `load_account` · `configured_address` · `account_created` · `check_address`

---

bench/core/networks.py A3 · A4 · C1

Every endpoint, chain ID, explorer URL and L1 contract address, loaded from YAML and never inline in code, so a run is described entirely by committed config. `connect_verified` refuses to proceed when an endpoint reports a different chain than the config claims — an endpoint quietly pointing at the wrong chain produces plausible results describing the wrong system.

`Network` · `load_networks` · `funding_priority` · `connect` · `connect_verified`

---

bench/core/workloads.py B1

Turns a workload definition into a transaction dict. Enforces the rule that **no selector and no calldata is ever written by hand**: selectors are derived from the committed ABI and arguments go through the ABI encoder. Gas is deliberately not set here, so that an estimation failure is reported as an estimation failure rather than becoming a hardcoded limit.

`Workload` · `load_workloads` · `recipient` · `token_address` · `selector` · `signature` · `build`

---

bench/core/submit.py B2 · B3 · B4 · B5

Signing, broadcasting, nonce assignment, receipt polling and outcome classification. Captures t_0 immediately before the broadcast. Raises on submission failure so the caller can record it with `hash` left null — **a fabricated hash is the single most damaging thing that can be done to a benchmark**, because every downstream measurement then looks plausible and means nothing.

`to_hex` · `estimate_gas` · `assign_nonces` · `verify_nonce_advance` · `submit` · `rejected_record` · `resolve_t1_batch` · `annotate_block_times` · `summarise` · `group_failures`

---

bench/core/records.py B2 · C5 · C6 · D1

The row schema and the JSON Lines file format. Defines the outcome vocabulary — success, reverted, timeout, rejected — and normalises rows on load so a file written before a field existed still reads as a rectangular table. Writes via a temporary file and an atomic move, so an interrupted pass cannot leave a half-written run file.

`Outcome` · `new_record` · `run_path` · `append` · `normalise` · `load` · `rewrite` · `iter_runs`

---

bench/core/settle.py C2 · C3 · C4 · C5

Turns L1 settlement hashes into timestamps: fetch the Ethereum transaction, read its block number, fetch that block, take its timestamp. Caches per L1 transaction, because a batch holds many of our rows and re-fetching each block once per row multiplies the request count for no new information. `outstanding()` decides what work remains.

`l1_timestamp` · `apply_settlement` · `outstanding`

---

bench/core/costs.py D2 · D3

Cost and data-availability size from L1 receipts — `gasUsed × effectiveGasPrice`, never a constant and never an estimate. Distinguishes blob-carrying transactions (type 3, priced at 131,072 bytes per blob) from calldata postings (16 gas per non-zero byte). Bytes and cost travel as separate fields because the byte count is a fact about the rollup while the cost depends on our price assumptions.

`L1Cost` · `l1_cost` · `pricing` · `to_usd`

Adapters — the only per-rollup code

bench/adapters/base.py C2 · E1

Defines `Settlement`, the one shape of answer every architecture returns, so the resolve pass never branches on which rollup it is talking to. Carries `t3_kind` — observed or derived — because a ZK rollup *witnesses* a proof while an optimistic rollup *computes* a deadline, and presenting the second as the first would be the most misleading thing in the study.

`Settlement` · `Adapter` · `SettlementUnavailable` · `OBSERVED` · `DERIVED`

---

bench/adapters/zksync.py C2

Resolves settlement via the L2 receipt's batch number and `zks_getL1BatchDetails`, caching per batch. Also yields the batch's own transaction count, which is the denominator the cost model needs.

`ZkSyncAdapter` · `batch_details` · `settlement`

---

bench/adapters/optimism.py E1 · E2

Reconstructs settlement for OP Stack chains, which expose none of what zkSync hands over. t_2 is found by reading the L2 block's L1 origin from the L1Block predeploy, anchoring by timestamp, and scanning forward for the batcher's posting — a real L1 transaction, matched by proximity rather than proven, and recorded as estimated. t_3 is computed from the dispute game's proposal time plus the challenge period read from the OptimismPortal, and recorded as derived.

`OptimismAdapter` · `l1_origin` · `block_at_or_after` · `scan_postings` · `find_batch_posting` · `challenge_period` · `game_covering` · `settlement`

---

bench/adapters/__init__.py C2 · E1

Maps a network's `family` to its adapter. Raises rather than returning a do-nothing default, because a rollup with no adapter would otherwise produce rows with no t_2 and no t_3 that look exactly like rows whose settlement has not happened yet.

`for_network`

Configuration — a run is described entirely by these

bench/configs/networks.yaml

Five chains: one L1 and four L2s. Endpoints, expected chain IDs, explorers, bridge URLs, faucet tiers, funding priority, and the L1 contract addresses two of the three timestamps are read from. Endpoints can be overridden per network from the environment (`BENCH_RPC_<KEY>`) to swap a public endpoint for a paid one.

---

bench/configs/workloads.yaml

Two workloads — a native transfer and an ERC-20 transfer — plus the fixed recipient and the deployed token address per network. Contains no selector and no calldata: those are derived from the ABI at runtime.

---

bench/configs/accounts.yaml

The test account's address, when it was created, and — recorded as it happens rather than remembered in Week 7 — which faucet or bridge funded each network. This is the report's experimental-setup section in raw form.

---

bench/configs/pricing.yaml

The ETH/USD rate, when it was taken, and from where. Read only at analysis time, never at collection time, so a wrong rate is a one-line fix and a re-export rather than a re-run. A dollar figure without its assumptions cannot be checked by anyone.

---

bench/abis/erc20.json · zksync_hyperchain.json

Committed rather than fetched at runtime, so results do not stop being reproducible because a third-party API is down or has changed its terms. The zkSync ABI carries the `BlockCommit` and `BlocksVerification` events the fallback route needs, and the batch counters mainnet observation will use.

---

bench/contracts/BenchToken.sol

The plainest possible OpenZeppelin ERC-20: no hooks, no pausing, no permit. The study measures what it costs a rollup to settle an ordinary token transfer, so anything unusual in this contract would appear in the gas and DA figures as though it were a property of the rollup. OpenZeppelin sources are vendored at a pinned 5.0.2 alongside it.

---

6 · What was learned by measuring

Each of these contradicted a reasonable expectation. They are recorded because they are the findings most likely to matter to somebody repeating this work.

The per-transaction settlement endpoint lags the per-batch one

The obvious route to settlement is `zks_getTransactionDetails`, which returns commit, prove and execute hashes for an L2 transaction. For batch 21623 it returned `ethProveTxHash: null` while `zks_getL1BatchDetails` for that same batch already carried a proof hash timestamped seven minutes after the commit. The transaction view catches up eventually, but any single pass over it **systematically undercounts trustless finality**, and nothing about the null value signals that it is stale rather than genuinely pending. Settlement is now read per batch.

The batch-posting transaction's recipient is not the rollup's contract

The address a commit transaction is sent to (`0xedf4B1C6…`) is the ValidatorTimelock — batches are submitted *through* it, and every state getter on it reverts. The contract holding batch state is a different address (`0x9a6de0f6…`), obtained from the chain itself via `zks_getMainContract`. A fallback route that queried logs on the timelock would find nothing and look like a broken query rather than a wrong address.

zkSync reverts on transfers to low addresses

The conventional burn address `0x…dEaD` is the integer 57,005, which falls inside the range zkSync reserves for system contracts. Transfers to it revert with a bare `execution reverted` and no reason. It estimates fine on Sepolia and fails only on the rollup — precisely the kind of thing that surfaces as an inexplicable Phase B failure. The recipient is now a deterministically derived address outside that range.

Gas estimates are not cost figures

On zkSync, the token deployment estimated 5,386,132 gas and used 590,512 — a ninefold overshoot. A transfer estimated 224,386 and used 79,259. On OP Sepolia the same deployment estimated 555,635 and used 550,102, which is accurate. Costs must come from receipts; an estimate-derived cost on zkSync would not be wrong by a little.

Batch size drives full-trust latency

Median t_1 was 0.61 s for a single transaction, 1.85 s for five, and 18.11 s for fifty. The first reading was that this must be an artefact of the polling interval — halving the poll cap moved the figure from 18.63 s to 17.51 s, which disproved it. It is the sequencer working through a queue of consecutive nonces from one account, and it is the effect the batch-size sweep exists to characterise.

Per-transaction data-availability size is not measurable here

A blob costs 131,072 bytes whether the rollup fills it or not, and batch 21623 carried 875 transactions of which 766 were not ours. So per-transaction DA bytes is a batch average identical across workloads, and the acceptance test for that task — DA bytes differing between a native and an ERC-20 batch — is **not reachable on this rollup at this scale**. The column is named `da_bytes_per_tx_batch_avg`, flagged `da_workload_sensitive=False`, and the limitation is stated in the export rather than hidden by a plausible-looking number. The workload difference is real and appears in L2 gas, where zkSync prices pubdata: 82,995 for a native transfer against 121,916 for an ERC-20 transfer.

An optimistic testnet does not shorten its challenge window

The implementation handbook states that testnets "use drastically shortened challenge windows — around an hour where mainnet takes seven days". Read directly from the OptimismPortal on Sepolia, OP Sepolia's `proofMaturityDelaySeconds` is 604800 — a full seven days, identical to mainnet — and the fault dispute game's `maxClockDuration` is 302400, three and a half days. Trustless finality on this rollup therefore **cannot be observed inside an eight-week project at all**, under any circumstances. It has to be computed and declared derived. That is a finding about the network rather than an obstacle to work around.

A settlement timestamp nine seconds before the transaction

Matching an OP Stack batch by scanning forward from the L2 block's L1 origin produced a t_2 that preceded t_0 — a batch posted before our transaction existed, which cannot possibly contain it. The cause is that the L1 origin lags the L1 head by a sequencer window, so the scan began in the past. The scan now anchors by timestamp, and a standing invariant refuses any settlement timestamp earlier than t_0 and says so. The invariant matters more than the fix: a future matching heuristic making the same class of mistake now fails loudly instead of producing a negative latency somebody has to notice by eye.

Our batch size is not the rollup's batch size — but the rollup's own is measurable

The batch-size sweep asks that per-transaction cost fall as batch size grows. It does not, and cannot be made to on a shared public rollup. Seven runs of sizes one through fifty all landed in zkSync batch 21623 alongside 875 other transactions and produced an identical per-transaction cost every time. Per-transaction cost is the batch's cost divided by the batch's own transaction count, and **that count is not something we control** — the rollup batches everybody's traffic on its own schedule and ours is a minority of it. The latency half of the sweep is real, because queueing is an effect we do cause. The honest form of the cost question is per-transaction cost against the *rollup's* batch size across many batches — and once eight batches had settled, that answered it cleanly. From 617 to 1,737 transactions per batch, per-transaction cost falls **66%** and tracks 1/n closely. The relationship the task asked about is real; it simply is not something we cause, so it had to be observed rather than induced. Same structural reason the data-availability measurement failed, and the same remedy.

A collection loop that looked like it was working

The first repetition of every matrix cell ran, and then nothing was going to run repetitions two through five. The matrix would have rested at 11 of 60 indefinitely while the plan output kept reporting "next due in 50 min" — the appearance of progress with no mechanism behind it. Unattended collection is now an hourly cron tick that submits at most one overdue cell. Worth recording because the failure mode was invisible from the inside: every individual component was working correctly.

Full-trust latency depends on what you mean by it

Measured t_1 exceeds the L2 block's own timestamp by a median of 17.95 s for batched submissions. The sequencer timestamps the block near submission but does not expose the receipt for some seconds afterwards. So t_1 as measured means "when the sequencer would tell us", not "when the sequencer says it happened" — an upper bound. Both are recorded so the gap is visible.

7 · Rules the code enforces

- **Never invent a number.** A field that cannot be measured is reported as unavailable. A submission that fails produces a row with a null hash, never a placeholder.

- **Never publish a cost without its assumptions.** The ETH rate, its source, and the L1 gas price appear beside every figure.
- **Never report a latency without its finality level.**
- **Raw per-transaction data is primary.** Summaries are derived and regenerable; they never replace the rows.
- **Absolute timestamps only.** No duration is stored at collection time.
- **Observed and derived values are distinguished**, because a computed challenge-window deadline is not a witnessed event.
- **Do not stress-test a shared public network.** Achieved throughput is reported; no peak figure is claimed.

8 · What comes next

Phases A through E are complete and the analysis figures are built. What is left divides cleanly into work that is blocked by time, work that is blocked by nothing, and work that belongs to the author.

What remains, and what is blocking each

TASK	BLOCKED BY
F2 repetitions	Wall clock, deliberately. 48 outstanding at one per hour; running them faster would measure a single moment repeatedly, which is the thing repeating exists to avoid
F3 batch sweep	Half of it is not reachable at all — see above. The latency half is done
G4 reproducibility	The calendar. Its test is "re-run on a different day"
G5 threats to validity	Nothing. It is simply unwritten, and most of its content is already scattered through section 6 of this document
H1–H5	The author. H1 needs papers actually opened; the report, presentation and handover are the intern's to write

A limit worth planning around

Because OP Sepolia's challenge window is a full seven days, the optimistic side of this study can never contribute an *observed* trustless timestamp. Every t_3 on that side will be derived for the lifetime of the project. That is not a gap to close but a property to report: it is why the comparison table carries a kind column, and it is itself part of the finding — the trustless guarantee an optimistic rollup offers is not something a user can watch arrive.

Outstanding decisions that need the supervisor

The project plan refers to a *"modular research testbed already designed and developed in our laboratory for ZK rollup technology"*. This implementation is deliberately observational and hosts nothing. Those are different approaches, and the difference has a concrete consequence: the plan asks for evaluation *"under varying load conditions"*, but a shared public sequencer must not be saturated — doing so would degrade a service other people depend on and would measure our own rate limits rather than the system. On public testnets the only load variable under our control is batch size. A lab testbed would remove that constraint. This is worth settling early rather than in Week 10.